Lesson 4.1 — Live Pair Programming with AI

Lesson 4.1 — Live Pair Programming with AI

> **Picture it.** You share your screen. The interviewer says: *"Implement a function that does X. Use whatever tools you normally use, including AI. Narrate as you go."* You have 45 minutes. The interviewer is silent. **What do you do for the first 60 seconds?**

This lesson answers that question.

By the end:

- You will have a step-by-step playbook for the first five minutes of any live AI-permitted coding interview.
- You will know the **narration moves** that signal seniority without sounding rehearsed.
- You will know the four **slow-down moments** when speed will hurt you.
- You will know how to handle the AI giving you bad output in front of a panel without losing composure.

---

1. The format you are walking into

In 2026, technical interviews increasingly come in three new flavors:

1.1. AI-Permitted Live Coding

> *"Use any tool you want. Narrate."*

The interviewer wants to see your **process**, not just the answer. They want to know:

- Do you read the problem before prompting?

- Do you write small prompts or large ones?
- Do you read the diff before accepting?
- Do you run the code?
- Can you explain what the AI gave you?

This is the most common 2026 format. **The rest of this lesson is mostly about it.**

1.2. AI-Forbidden Live Coding

> *"No AI for this one — just you and the editor."*

Less common but increasing again as a counter-trend. The interviewer wants a baseline of your unaided coding ability. **Refusing this format because "I always use AI" is a red flag.** Be ready for both.

1.3. AI-Augmented Take-Home

> *"Build this small project. AI is fine. Submit a PR within 48 hours plus a 5-minute Loom walking through your decisions."*

The Loom is the interview. Spend twice as much time on the Loom as you spend on the code.

---

2. The first 60 seconds — the playbook

When the interviewer goes silent, you do this. Out loud.

Step 1 (5–10 seconds): Re-state the problem

> *"OK, so you want a function that takes [input] and returns [output], and the constraints are [constraint 1] and [constraint 2]. Did I understand correctly?"*

This buys you thinking time and **forces a clarification before you write code.** Half the time the interviewer says "actually, also…" and you just dodged a wasted 10 minutes.

Step 2 (10–20 seconds): Ask one or two clarifying questions

Pick ones that change the design.

"Should this handle the empty input case as zero or as an error?"

"Should I prioritize readability or performance for this scale?"

"Are there existing patterns in the codebase I should follow, or is this a greenfield decision?"

The questions matter less than the **fact** that you asked. Junior candidates start typing in 5 seconds. Seniors talk for 30 seconds before any code happens.

Step 3 (20–40 seconds): State your approach in plain English

"My plan is: define the type first, then write a pure function that handles the happy path, then add the edge cases I just asked about, then write three unit tests covering those cases."

You are essentially **writing the prompt structure out loud** before opening the AI panel. The interviewer immediately sees: this person thinks in structure.

Step 4 (40–60 seconds): Open the AI tool, write the first prompt deliberately

Do not paste. Type. Out loud, narrate the components from Lesson 3.1:

"OK, my prompt: 'Define a TypeScript interface for [thing]. Strict — no any. Add a comment explaining the invariant about [whatever]. Single file. No tests yet.'"

The interviewer sees you naming components like *strict*, *invariant*, *single file*, *no tests yet*. Each phrase is a senior signal.

Press Enter at the 60-second mark. Now you are coding.

---

3. The narration framework — three rhythms

You will narrate for the entire interview. Three rhythms work; everything else sounds like a podcast or a panic.

3.1. The pre-prompt narration

Before each prompt, narrate what you are about to ask and why.

> *"Now I want to add a render function. I'll prompt for it explicitly — I'll keep it small and tell it not to add tests yet, I'll write those myself."*

Time: ~5 seconds. Frequency: every prompt.

3.2. The diff-review narration

When the AI returns output, narrate as you read.

> *"OK, it added the function. The signature looks right. The body… iterates the array, accumulates… yep, that handles the happy path. But it does not handle the empty input case I asked about. I will reject this and re-prompt."*

Time: 10–30 seconds. Frequency: every diff.

This is the most senior-coded behavior in the entire interview. **You read AI output out loud, criticize it, and decide.** That is engineering judgment on display.

3.3. The decision narration

When you make a design choice, narrate it.

> *"I have two options here: keep the helper inline or extract it. It's only used once. I'll keep it inline for now — I will extract if a second use case appears."*

Time: 5–15 seconds. Frequency: ~once every five minutes.

These are the moments interviewers note for the hiring committee. Decisions, narrated, with reasons.

---

4. Pacing — the four slow-down moments

Most candidates rush. The four moments where slowing down will help you, not hurt you:

4.1. After the prompt produces output

Read it line by line. Out loud. Two of three candidates skim and accept; you read and verify. **The interviewer's notes will reflect that.**

4.2. Before accepting a refactor

If the AI is changing existing code, *always* slow down. *"Let me read the diff before I accept."* The interviewer hears: this person catches accidental changes. Senior signal.

4.3. Right before you say "I think we are done"

Take 30 seconds. Re-read the requirement. Walk through one input mentally. **Catch one off-by-one or null-handling miss before the interviewer does.** Often the difference between an offer and a "not quite this round."

4.4. When the interviewer says "Hmm…"

If the interviewer makes a hesitant noise, **stop typing**. Look up. Say: *"You sound like you might be seeing something. Want to share?"* This converts a potential pothole into a teaching moment for them and survives many close calls.

---

5. Prompt hygiene under observation

In the calm of your home you write good prompts. Under observation, you regress. Two specific failure modes:

5.1. The "do everything" panic prompt

Under stress, candidates write:

> *"Build me a function that handles all the cases including edge cases and add tests and documentation and make it efficient."*

That is four prompts mashed into one. The model gives mediocre output across all four. **Force yourself to keep prompts small even — especially — under stress.**

5.2. The "skip the constraint" speed-prompt

You skip the constraints because you want to move fast. The output sprawls. You spend 10 minutes cleaning up. **Net negative.**

5.3. The fix

Before each prompt under observation, count to three in your head. *"Three. Two. One."* Three seconds of breathing space between thinking and typing. **It feels long. It is fast.**

---

6. Handling bad AI output in front of a panel

The AI will, occasionally, give you bad output during the interview. The interviewer is watching how you handle it. **This is a feature of the interview, not a bug.**

What NOT to do

- Apologize for the AI. *"Sorry, the AI is being weird."* Sounds like deflection.
- Argue with the AI. *"No, I told you not to…"* — pointless on screen.
- Panic-accept. The diff is wrong but you accept it because you do not know what to do.
- Switch to typing it yourself silently. The narration died; the interviewer is now staring at your hands.

What to do

1. Pause. *"OK, this is not what I asked for."*
2. Diagnose out loud. *"It overshot — refactored the parent. I will reject this and re-prompt with explicit scope."*
3. Re-prompt. Narrate the new prompt's added constraint.
4. Move on.

The recovery is the demonstration. A panel watching a candidate handle bad AI output cleanly is more impressed than a panel watching a candidate get clean AI output on the first try. Mastery is recoverability.

---

7. The "you have 45 minutes, build a small feature" worked example

Pretend the interviewer says:

> *"Build a function that takes a list of users and returns the top 3 by signup recency, only including users who have verified emails. Plus three tests. You have 45 minutes."*

A senior approach, narrated:

Minute 0:00–1:00 — Re-state and clarify

> *"So I need a function: input is a list of users, output is the top 3 by signup recency, filtering out unverified. Two questions: how is signup recency stored — timestamp, ISO string, Date? And if there are fewer than 3 verified users, do I return what I have or throw?"*

Interviewer: *"ISO string. Return what you have."*

Minute 1:00–3:00 — Plan and type definition

Open AI. Type:

> *"Generate a TypeScript interface for User with id (string), email (string), signupAt (ISO string), emailVerified (boolean). Strict mode. Single file. No tests."*

Read the diff. Accept. Narrate that you accepted because the shape is exactly what you specified.

Minute 3:00–10:00 — Pure function

Prompt:

> *"Write a pure function getTopRecentVerifiedUsers(users: User[], limit: number): User[]. Filters to emailVerified === true, sorts by signupAt descending (parse ISO string with Date.parse), returns the first `limit`. If fewer than `limit` qualify, return what we have. Single file. No tests yet."*

Read the diff. Spot something — say, the AI used `new Date()` and could have used `Date.parse()`. Decide: not worth the round-trip; both are correct. Accept and narrate the decision.

Minute 10:00–17:00 — Tests

Prompt:

> *"Write three Vitest tests for getTopRecentVerifiedUsers. Cases: (1) returns 3 users sorted newest-first when 5 verified candidates exist; (2) returns only 2 when only 2 verified candidates exist; (3) returns empty array when input is empty. Use describe/it. One assertion each."*

Run the tests. Two pass, one fails. Look at the failure.

Minute 17:00–25:00 — Debug

The failing test: returns 3 users newest-first. Debug. The output is correct in count but the order is *oldest-first*. Look at the function. The sort is wrong direction.

Narrate:

> *"OK, the AI sorted ascending instead of descending. I'll patch the comparator. One-line fix."*

Edit yourself. Re-run. All green.

Minute 25:00–35:00 — Hardening

Run a tooth-comb pass:

- *"What if signupAt is malformed?"* — add a guard.
- *"What if limit is 0 or negative?"* — add a guard, or document that it returns empty.
- *"What if users have duplicate signupAt timestamps?"* — note that JS sort is stable, so existing order wins. Mention it; do not change.

Each guard is a 30-second narration. The interviewer is watching you raise the bar yourself.

Minute 35:00–45:00 — Trade-offs

Stop typing. Narrate:

> *"With time left, here are three things I would want to discuss before shipping this. One, performance — if `users` is large, the .filter().sort().slice() is O(n log n); for large n we could use a partial sort, but I would benchmark first. Two, testability — the function uses Date.parse, which depends on locale; for production I would inject a clock or a parser. Three, naming — `getTopRecentVerifiedUsers` is descriptive but long; in a real codebase I might check naming conventions."*

That ten minutes is **the part that gets you the offer.** Most candidates ship the function and stop. You ship the function and *then talk about what you would do next*.

---

8. The "I have not heard them speak in 10 minutes" awkwardness

Common in remote interviews. The interviewer goes silent. You wonder if they're judging you. Resist the urge to fill the silence with random chatter.

Two tools

Checkpoint questions, every 5 minutes.

> *"OK, before I move on — does this approach feel right to you, or would you want to see something different?"*

Verbal labels for your steps.

> *"Now moving on to tests. I'll write three."*

Both keep them oriented and signal that you are aware of pacing.

---

9. The closing 60 seconds

When time is up — or when you think you're done — say something like:

> *"OK, I think this is a good stopping point. Quick summary: I built [function], tested it for [cases], and identified [N] things I'd want to address before shipping. The biggest of those is [one]. Want me to dig into any part of this?"*

That summary:

- Confirms what you finished.
- Shows you know what was *not* finished and what should be next.
- Hands the floor back to them with a specific question.

It is the closing senior move of any technical interview.

---

10. The ten-item live-coding checklist

Print this. Stick it next to your monitor for practice sessions.

- ☐ Re-state the problem before typing.
- ☐ Ask 1–2 clarifying questions.
- ☐ State your approach in plain English.
- ☐ Write a strict, scoped first prompt.
- ☐ Read every diff out loud before accepting.
- ☐ Reject any output that overshoots scope.
- ☐ Run the code; do not trust the AI's claim that it works.
- ☐ Test the actual edge cases the spec named.
- ☐ Spend the last 10 minutes on hardening and trade-offs.
- ☐ Close with a 30-second summary.

Practice with this checklist visible until each item is automatic.

---

11. Practice exercise — record three interviews

The single highest-ROI thing in this chapter is recording yourself.

1. Pick three small problems (find any LeetCode "easy" problem set, or use the snooker-app refactors from Chapter 7 of the React course).
2. Solve each one under timed conditions, screen-recording, narrating.
3. Watch the recording back at 1.5x.

You will spot:

- Filler words you did not know you used.
- Long silences that felt fine in the moment.
- Prompts that were vaguer than you remembered.
- Diffs you accepted without reading carefully.

One week of this beats six weeks of reading.

---

12. Common pitfalls

12.1. Over-narration

Talking nonstop. The interviewer cannot think. **Pause every 30–60 seconds**. Silence is allowed.

12.2. Under-narration

Typing silently. The interviewer cannot read your process. **Default to narrating each prompt and each diff review.**

12.3. Selling the AI instead of yourself

Saying things like *"watch how fast this is with AI"*. The interview is about you, not the tool. **Talk about your decisions; the AI is in the background.**

12.4. Hiding the AI

Some candidates open the AI panel offscreen and quietly use it. **The interviewer notices.** Be visible. The AI is part of your toolset; do not be secretive.

---

13. CHECK YOURSELF

- ☐ Can you list the four steps of the first 60 seconds from memory?
- ☐ Can you list the three narration rhythms?
- ☐ Can you list the four slow-down moments?
- ☐ Have you recorded yourself solving at least one timed problem?
- ☐ Can you handle a deliberately bad AI diff cleanly without panic?

---

14. Where you are now

You have the playbook for the most common technical interview format of 2026. Move on to **02-debugging-ai-bugs.md** — the second-most-common format, where the interviewer hands you broken AI-generated code and watches how you fix it.